Fundamentos de Kubernetes y Arquitectura

Unidad 3

Kubernetes • Componentes • Control Plane • API Server • Manifiestos YAML

Curso de Contenedores y Orquestación

Agenda

Contenido de la Unidad 3

¿Qué es Kubernetes? — 01
Orquestador de contenedores, historia y por qué usarlo

Componentes de Kubernetes — 02
Arquitectura: Control Plane y Worker Nodes

Control Plane en Detalle — 03
API Server, etcd, Scheduler, Controller Manager

Flujo de Despliegue — 04
Desde kubectl apply hasta el Pod corriendo

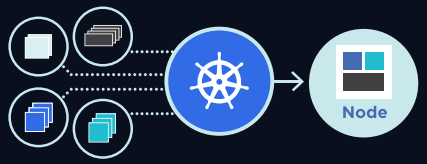
Riesgos del API Server — 05
Superficie de ataque, RBAC y mitigaciones

Manifiestos YAML + Labs — 06
Pods, Deployments, Services — desplegados en servidor Linux

¿Qué es Kubernets?

Plataforma open-source de orquestación de contenedores

Kubernets (K8s) es una plataforma de código abierto para automatizar la implementación, el escalado y la administración de aplicaciones en contenedores. Nacio de un Proyecto interno de Google y fue donado a la CNCF (Cloud Native Computing Foundation) en 2015.



Autoescalado

Escala automáticamente los Pods según CPU, memoria o métricas personalizadas.

Self-Healing

Si un contenedor falla, K8s lo reinicia automáticamente. Si un nodo cae, reubica los Pods.

Service Discovery

Asigna DNS y balanceo de carga a cada servicio sin configuración manual.

Rollouts y Rollbacks

Despliega nuevas versiones gradualmente. Si falla, revierte al estado anterior.

Gestión de Volumenes

Monta automáticamente almacenamiento local, cloud (EBS, GCE PD) o NFS.

Gestión de Secretos

Kubernetes te permiten almacenar y administrar información confidencial, como contraseñas, tokens OAuth y llaves ssh.

Docker vs Kubernetes

No compiten, se complementan

Docker

- Crea y ejecuta contenedores individuales
- Empaqueta aplicaciones en imágenes
- Ideal para desarrollo local
- Un contenedor en una máquina
- Dockerfile → Image → Container
- Sin orquestación nativa

Kubernetes

- Orquesta cientos/miles de contenedores
- Gestiona despliegues, red y escalado
- Ideal para producción y equipos
- Múltiples contenedores en múltiples nodos
- Manifest → API → Scheduler → Pod
- Orquestación completa del cluster

Docker construye los contenedores. Kubernetes los orquesta en producción. Se usan juntos, no uno en vez del otro.

Arquitectura de Kubernetes

Un cluster tiene Control Plane (cerebro) y Worker Nodes (músculos)

CONTROL PLANE (Master)

API Server

Puerta de entrada al cluster. Recibe todas las peticiones.

etcd

Base de datos del cluster. Almacena todo el estado.

Scheduler

Decide en qué nodo correr cada Pod.

Controller Manager

Ejecuta controladores que mantienen el estado deseado del cluster (ReplicaSet Controller, Node Controller, Endpoint Controller)

→

WORKER NODES

Kubelet

Agente que corre en cada nodo “supervisor”. Recibe instrucciones del API Server y gestiona Pods.

kube-proxy

Gestiona las reglas de red, permitiendo la comunicacion entre los servicios tanto internos como externos

Container Runtime

Ejecuta los contenedores (containerd, CRI-O, docker).

Workloads

Es una aplicación que corre en kubernetes.

kubectl — CLI que habla con el API Server

Analogía Visual del Clúster

Los componentes que toman las decisiones del cluster

Control plane = El cerebro

Toma decisiones, planifica y coordina todas las operaciones del clúster

Worker Nodes = Los brazos

Ejecutan el trabajo real, corren las aplicaciones y procesan las cargas

Api server = El sistema nervioso

Transmite las órdenes del cerebro a los brazos y reporta el estado de vuelta

ETCD = La memoria

Almacena todo lo que el cerebro necesita recordar sobre el estado del clúster

Flujo de Despliegue de una Aplicación

¿Qué pasa cuando ejecutas kubectl apply?

1 — kubectl apply -f deploy.yaml
El usuario envía el manifiesto al API Server

→

2 — API Server valida y guarda
Autentica, autoriza, valida el YAML y lo persiste en etcd

→

3 — Controller Manager actúa
Detecta que falta un ReplicaSet y crea los Pods necesarios

→

4 — Scheduler asigna nodo
Evalúa recursos y elige el mejor nodo para cada Pod

→

5 — kubelet ejecuta el Pod
El agente del nodo descarga la imagen y levanta los contenedores

→

6 — Pod Running ✓
La aplicación está corriendo y el Service expone el tráfico

Ejemplo completo: desplegar Nginx en el servidor

```
# 1. Conectarse al servidor Linux por SSH
ssh usuario@<IP_DEL_SERVIDOR>

# 2. Crear directorio de trabajo y escribir el manifiesto
mkdir -p ~/k8s-labs && cd ~/k8s-labs
vim nginx-deployment.yaml

# 3. Aplicar al cluster
kubectl apply -f nginx-deployment.yaml
# deployment.apps/nginx created

# 4. Verificar el estado
kubectl get pods -w
# NAME                    READY   STATUS    RESTARTS   AGE
# nginx-7bf8c77b5b-x4r2k  1/1     Running   0          10s
```

Objetos Principales de Kubernetes

Los recursos fundamentales que componen una aplicación

Pod

Unidad mínima. Uno o más contenedores que comparten red y storage. Efímero: si muere, no se recrea solo.

ReplicaSet

Garantiza N réplicas de un Pod. Si un Pod muere, crea otro. Raramente se usa directo (lo gestiona Deployment).

Deployment

El objeto más usado en K8s. Gestiona ReplicaSets y permite actualizaciones declarativas de las aplicaciones.

Statefulsets

Gestionan aplicaciones con estado que requieren identidad de red estable y almacenamiento persistente. Aplicado a bases de datos como MySQL, PostgreSQL, MongoDB en Kubernetes.

Secret

Almacena datos sensibles (passwords, tokens, certificados) codificados en base64.

ConfigMap

Almacena datos de configuración no confidenciales en pares clave-valor. Como variables de entorno, archivos de config, argumentos CLI.

Objetos Principales de Kubernetes

Los recursos fundamentales que componen una aplicación

Namespace

Partición lógica del clúster para aislar recursos entre equipos o entornos. Ej: dev, staging, production.

Service

Punto de acceso estable a un conjunto de Pods. Service proporciona red estable para pods (que son efímeros y pueden cambiar de IP).

Volumenes

Proporcionan almacenamiento persistente a los pods, sobreviven a reinicios de contenedores. Tipos principales: emptyDir, hostPath, PersistentVolume (PV), PersistentVolumeClaim (PVC)

RBAC

(Role-Based Access Control) controla quién puede hacer qué en el clúster mediante roles y bindings.

Ingress

Gestiona el acceso HTTP/HTTPS externo a los servicios del clúster. Nos brinda un punto de entrada para tráfico externo al clúster.

Anatomía de un Manifiesto YAML DEPLOYMENT

Todo en Kubernetes se define con manifiestos declarativos

Pod básico — manifiesto.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mi-nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.25-alpine
    ports:
    - containerPort: 80
    resources:
      limits:
        memory: "128Mi"
        cpu: "250m"
```

Campos obligatorios

apiVersion

Versión de la API de K8s para este recurso. Ej: v1, apps/v1, networking.k8s.io/v1

kind

Tipo de objeto: Pod, Deployment, Service, ConfigMap, Secret, etc.

metadata

Identificación: nombre, namespace, labels y annotations del recurso.

spec

Estado deseado: contenedores, puertos, volúmenes, réplicas, etc.

containers

Lista de contenedores del Pod. Cada uno tiene: name, image, ports, resources.

resources

Límites de CPU y memoria. Evitan que un Pod consuma todos los recursos del nodo.

Manifiesto: Deployment

El recurso más importante — gestiona el ciclo de vida de los Pods

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25-alpine
        ports:
        - containerPort: 80
```

¿Qué hace el Deployment?

replicas: 3

Mantiene siempre 3 Pods corriendo. Si uno muere, crea otro automáticamente.

selector + labels

Conecta el Deployment con sus Pods. El selector busca Pods con label app: nginx.

Rolling Update

Al cambiar la imagen, K8s crea Pods nuevos y elimina los viejos gradualmente. Zero downtime.

Manifiesto: Service

Expone tus Pods al mundo — punto de acceso estable

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

Tipos de Service

ClusterIP (default)

Solo accesible dentro del cluster. Ideal para comunicación entre microservicios internos.

NodePort

Expone el servicio en un puerto del nodo (30000-32767). Accesible: http://<IP_SERVIDOR>:<NodePort>

LoadBalancer

Crea un balanceador en la nube (AWS ELB, GCP LB). Asigna IP pública automáticamente.

El Service encuentra sus Pods mediante el selector (labels). En este caso, selector: app: nginx conecta con todos los Pods que tengan la label app: nginx. Si escalas a 5 réplicas, el Service balancea automáticamente el tráfico entre los 5 Pods. Con NodePort accedes desde: http://<IP_SERVIDOR>:30080

Riesgos del API Server

Superficie de ataque y vectores comunes

El API Server es el componente más crítico. Si un atacante obtiene acceso, controla TODO el cluster.

API Expuesto a Internet

Si el puerto 6443 está abierto sin restricción, cualquiera puede intentar autenticarse. Scanners automatizados buscan clusters K8s expuestos.

RBAC Mal Configurado

Permisos excesivos (ClusterAdmin a todos) permiten que un usuario comprometa todo. Principio de mínimo privilegio no aplicado.

Tokens de ServiceAccount

Cada Pod recibe un token automáticamente. Si un atacante entra a un Pod, puede usar ese token para hablar con el API Server.

Secrets en Texto Plano

Por defecto, los Secrets en etcd no están encriptados. Si se accede a etcd, se leen todas las contraseñas y tokens.

Pods Privilegiados

Contenedores corriendo como root o con **privileged: true** pueden escapar al nodo host y comprometer el cluster.

Sin Audit Logging

Sin logs de auditoría no se detectan accesos no autorizados ni se puede hacer forensics post-incidente.

Seguridad del API Server

Mitigaciones y buenas prácticas

RBAC (Role-Based Access Control)

Definir roles con permisos específicos por namespace. Nunca usar cluster-admin en producción. Crear roles granulares: solo listar pods, solo leer secrets de un namespace.

Network Policies

Restringir qué Pods pueden comunicarse entre sí y quién puede alcanzar el API Server. Usar Calico o Cilium como CNI para enforcement.

Encriptar etcd y Secrets

Habilitar encryption at rest para etcd. Usar un KMS (Key Management Service) externo para gestionar las claves de encriptación.

ServiceAccount mínimo

Deshabilitar automountServiceAccountToken en Pods que no lo necesitan. Crear ServiceAccounts dedicados con permisos mínimos. **

Audit Logging

Habilitar audit policy en el API Server para registrar quién hizo qué y cuándo. Enviar logs a un SIEM para alertas. Habilitado por defecto

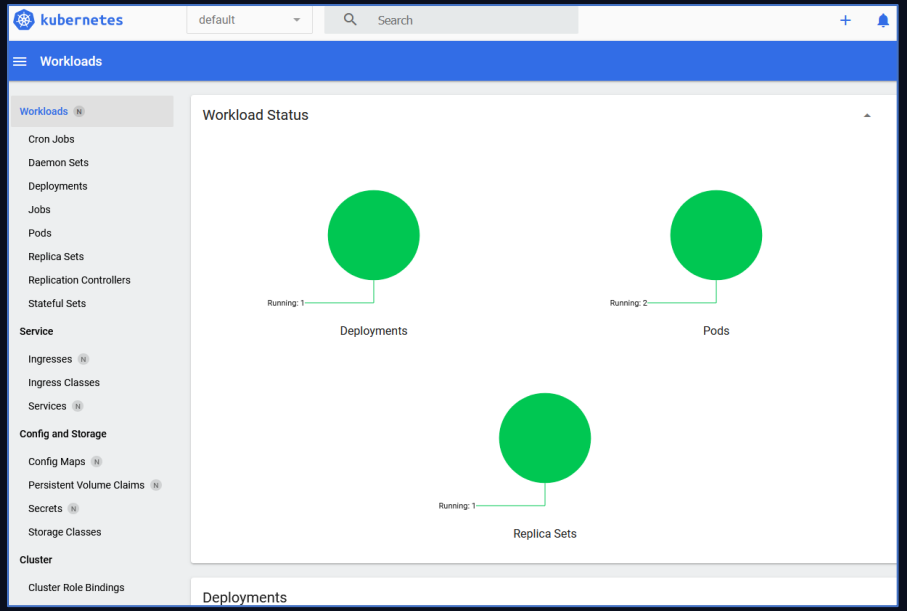
Pod Security Standards

Usar Pod Security Admission (PSA) para prevenir Pods privilegiados, root, y hostPath. Tres niveles: privileged, baseline, restricted. *** Pipeline release. ***

Acceso al Dashboard de Kubernetes

Acceder al panel de control del clúster mediante Token de autenticación

El Dashboard es la interfaz web de Kubernetes. Se comunica con el **API Server** para visualizar y gestionar recursos del clúster (Pods, Deployments, Services, Secrets).



Despliegue paso a paso

```
# Conectarse al servidor
ssh usuario@<IP_SERVIDOR>

# Verificar que los pods del Dashboard estén corriendo
kubectl get pods -n kubernetes-dashboard
# NAME                                    READY
STATUS    AGE
# kubernetes-dashboard-xxxx               1/1
Running

# Acceder desde navegador
# http://<IP_SERVIDOR>:65500
```

IMPORTANTE: Las IPs públicas de los participantes deben ser registradas previamente para permitir el acceso al Dashboard.

kubectl — Comandos Esenciales

La herramienta CLI para interactuar con el cluster

Información y Estado

`kubectl cluster-info`

Ver info del cluster

`kubectl get nodes`

Listar nodos del cluster

`kubectl get pods`

Listar Pods en el namespace actual

`kubectl get all`

Listar todos los recursos

`kubectl describe pod <n>`

Detalles completos de un Pod

`kubectl logs <pod>`

Ver logs de un contenedor

Crear y Gestionar

`kubectl apply -f archivo.yaml`

Crear/actualizar recursos

`kubectl delete -f archivo.yaml`

Eliminar recursos del YAML

`kubectl scale deploy <n> --replicas=5`

Escalar a 5 réplicas

`kubectl exec -it <pod> -- /bin/sh`

Abrir shell en un Pod

`kubectl port-forward pod/<n> 8080:80`

Reenviar puerto local

`kubectl rollout undo deploy/<n>`

Rollback a versión anterior

Laboratorio Práctico

3 Labs — Manifiestos YAML en Servidor Linux

01

Nginx con Pod y Service

```
lab1-nginx-pod.yaml
lab1-nginx-service.yaml
```

02

App con Deployment + Réplicas

```
lab2-app-deployment.yaml
lab2-app-service.yaml
```

03

Multi-Tier: Frontend + Redis

```
lab3-redis-*.yaml
lab3-frontend-*.yaml
```

Lab 1: Nginx - Pod

Mi primer manifiesto: desplegar Nginx en el cluster

lab1-nginx-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-lab1
  labels:
    app: nginx-lab1
spec:
  containers:
  - name: nginx
    image: nginx:1.25-alpine
    ports:
    - containerPort: 80
    resources:
      limits:
        memory: "64Mi"
        cpu: "100m"
```

Despliegue paso a paso

```bash
# Conectarse al servidor
ssh usuario@<IP_SERVIDOR>
cd workshop/unidad3/lab1

# Crear el Pod
kubectl apply -f lab1-nginx-pod.yaml

# Ver estado del Pod
kubectl get pods
# NAME         READY  STATUS   AGE
# nginx-lab1   1/1    Running  5s
```

Lab 1: Nginx Service y Verificación

Exponer Nginx fuera del cluster y verificar el despliegue

lab1-nginx-service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-lab1-svc
spec:
  type: NodePort
  selector:
    app: nginx-lab1
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

Despliegue paso a paso

```
# Crear Service (NodePort)
kubectl apply -f lab1-nginx-service.yaml

# Ver el Service
kubectl get svc nginx-lab1-svc
# nginx-lab1-svc   NodePort   80:30080/TCP
```

Verificación y limpieza

```
# Verificar acceso desde el servidor
curl http://IP_SERVIDOR:30080
# curl http://<IP_SERVIDOR>:30080

# Eliminar los recursos del Lab 1
kubectl delete -f lab1-nginx-pod.yaml
kubectl delete -f lab1-nginx-service.yaml

# Verificar que se eliminaron
kubectl get pods,svc
```

```
# Acceder desde navegador
# http://<IP_SERVIDOR>:30080

# Ver logs y entrar al Pod
kubectl logs nginx-lab1
```

Lab 2: Enumerar recursos

Pistas del reto

OBJETIVO:

1. Encontrar un Pod llamado **"paraiso"** en algún namespace del cluster

2. Identificar en qué Puerto está expuesto el Service

3. Conectarse desde el navegador e identificar la Flag{XXXXXXXXXX}

TIPS:

- No está en el namespace default
- Usa -A para buscar en todos

Comandos de ayuda

```
# Buscar pods en TODOS los namespaces
kubectl get pods -A

# Filtrar por nombre
kubectl get pods -A | grep paraiso

# Ver servicios del namespace
kubectl get svc -A

# Describir el pod para ver detalles
kubectl describe pod paraiso \
  -n <NAMESPACE>

# Describir el service
kubectl describe svc paraiso-svc \
  -n <NAMESPACE>

# Conectarse desde el navegador
# http://<IP_SERVIDOR>:<PUERTO>
```

Lab 3: Deployment con Réplicas

Desplegar httpd con 2 réplicas y escalar dinámicamente

lab2-app-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-lab2
spec:
  replicas: 2
  selector:
    matchLabels:
      app: httpd-lab2
  template:
    metadata:
      labels:
        app: httpd-lab2
    spec:
      containers:
      - name: httpd
        image: httpd:2.4-alpine
        ports:
        - containerPort: 80
        resources:
          limits:
            memory: "64Mi"
            cpu: "100m"
```

Despliegue y escalado

```bash
# Aplicar Deployment (2 réplicas)
kubectl apply -f lab2-app-deployment.yaml

# Ver los 2 Pods creados
kubectl get pods -l app=httpd-lab2
# httpd-lab2-xxx-aaa  1/1  Running
# httpd-lab2-xxx-bbb  1/1  Running


# Eliminar un Pod (self-healing)
kubectl delete pod <NOMBRE_POD>
# K8s crea uno nuevo automáticamente!

# Eliminar los Pods definitivamente
kubectl delete -n default deployment httpd-lab2
```

Comandos Útiles — Unidad 3

Referencia rápida para Kubernetes en el servidor

Recursos y Estado

`kubectl get pods -o wide`

Pods con IP y nodo asignado

`kubectl get deploy,svc,pod`

Varios recursos a la vez

`kubectl describe deploy <n>`

Detalles del Deployment

`kubectl get events --sort-by=.metadata.creationTimestamp`

Eventos recientes del cluster

`kubectl top pods`

CPU y memoria de Pods

`kubectl explain deployment.spec`

Documentación inline de K8s

Debug y Operaciones

`kubectl logs <pod> -f`

Logs en tiempo real

`kubectl exec -it <pod> -- sh`

Shell en el contenedor

`kubectl port-forward svc/<n> 8080:80`

Reenviar puerto local

`kubectl rollout status deploy/<n>`

Estado del rollout

`kubectl rollout history deploy/<n>`

Historial de versiones

`kubectl delete all --all`

Eliminar todo (¡cuidado!)

Resumen — Unidad 3

- Kubernetes orquesta contenedores: autoescalado, self-healing, service discovery
- Arquitectura: Control Plane (cerebro) + Worker Nodes (ejecutan los Pods)
- API Server es la puerta de entrada — protegerlo con RBAC y Network Policies
- Manifiestos YAML definen el estado deseado: Pods, Deployments, Services
- Despliegue en servidor Linux: ssh → kubectl apply -f → curl para verificar